

# AI Career Counsellor

## Technical Build Specification

---

Internal Engineering Document

Smart India Hackathon 2 (SIH2) -- ai-career-counsellor

Version 1.0 -- August 17, 2026

Prepared for: Development Team

This document defines the full technical architecture, page-by-page design, database schema, AI system design, automation logic, and role-based access model for the platform. It consolidates every decision made in planning discussions into a single build reference.

# Table of Contents

Table of Contents

2

1. Document Purpose & Scope

5

1.1 Purpose . . . . .

5

1.2 Audience . . . . .

5

1.3 How to Use This Spec . . . . .

5

2. Project Overview

6

2.1 What We're Building . . . . .

6

2.2 Roles . . . . .

6

2.3 Current Build Status . . . . .

6

3. Technology Stack

7

3.1 Core Framework (already decided) . . . . .

7

3.2 Styling & UI System (already decided) . . . . .

7

3.3 Backend & Data Layer (already decided) . . . . .

7

3.4 AI Layer (new -- required for this spec) . . . . .

7

3.5 Automation & Integrations (new) . . . . .

8

3.6 Tooling (already decided) . . . . .

8

4. Information Architecture -- Pages & Layouts

9

4.1 Layout Templates . . . . .

9

4.2 Full Route Inventory . . . . .

9

4.3 Page-by-Page Breakdown . . . . .

9

5. Design System & Styling

12

5.1 Styling Approach (already decided) . . . . .

12

5.2 Typography . . . . .

12

5.3 Core Color Tokens (light mode) . . . . .

12

5.4 Component Library (already built) . . . . .

12

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 5.5 The Robot Assistant -- Technical Implementation . . . . . | 12        |
| 5.6 Motion & Animation Strategy . . . . .                     | 13        |
| 5.7 Responsive Strategy . . . . .                             | 13        |
| <b>6. Database Architecture</b>                               | <b>14</b> |
| 6.1 Entity Overview . . . . .                                 | 14        |
| 6.2 Full Prisma Schema . . . . .                              | 14        |
| 6.3 Field-Level Notes . . . . .                               | 16        |
| <b>7. AI System Architecture</b>                              | <b>18</b> |
| 7.1 Two AI Touchpoints . . . . .                              | 18        |
| 7.2 Assessment Recommendation Engine . . . . .                | 18        |
| 7.3 AI Chatbot -- Dual-Mode Design . . . . .                  | 19        |
| 7.4 Token & Usage Management . . . . .                        | 19        |
| <b>8. Automation Systems</b>                                  | <b>21</b> |
| 8.1 Automated PDF Report Generation . . . . .                 | 21        |
| 8.2 Mentor Matching Automation . . . . .                      | 21        |
| 8.3 Session Refresh Automation (already built) . . . . .      | 21        |
| <b>9. Role-Based Access Control</b>                           | <b>22</b> |
| 9.1 Role Storage Strategy . . . . .                           | 22        |
| 9.2 Route Protection Matrix . . . . .                         | 22        |
| 9.3 Middleware Flow . . . . .                                 | 22        |
| <b>10. Payments &amp; Premium System</b>                      | <b>23</b> |
| 10.1 Razorpay Integration Flow . . . . .                      | 23        |
| 10.2 Premium Feature Gate . . . . .                           | 23        |
| 10.3 Security Notes . . . . .                                 | 23        |
| <b>11. API Route Inventory</b>                                | <b>24</b> |
| <b>12. Folder &amp; File Structure</b>                        | <b>25</b> |

13. Voice Command System

Intent Mapping . . . . .

14. Development Roadmap

15. Developer Commands

16. Appendix -- Assumptions & Open Decisions

27  
27  
28  
30  
31

# 1. Document Purpose & Scope

---

## 1.1 Purpose

This document translates the product discussions for AI Career Counsellor into a concrete, buildable technical specification. It is written for the people building the system, not for hackathon judges -- it assumes familiarity with the existing codebase (Next.js 16 / Supabase / Prisma) already scaffolded in the repository.

## 1.2 Audience

Primary: the development team implementing the platform. Secondary: any contributor onboarding onto the project who needs full context without re-deriving decisions from chat history.

## 1.3 How to Use This Spec

- Sections 3-13 are the implementation reference -- stack, pages, design system, database, AI, automation, access control, payments, and API surface.
- Section 14 sequences the work into build phases.
- Section 16 lists every judgment call made on your behalf, so anything that doesn't match your intent can be revisited before implementation.

## 2. Project Overview

### 2.1 What We're Building

AI Career Counsellor is a role-based web platform with two roles: Student and Admin. A student completes an AI-driven aptitude and career assessment, receives a structured roadmap (career paths, recommended entrance exams, matched universities, competition and demand outlook), can chat with an AI assistant that becomes personalized once an assessment exists, and can be matched -- manually or automatically -- with a mentor. The Admin manages the mentor directory, reviews and confirms matches, and monitors subscription revenue.

### 2.2 Roles

| Role    | Who                      | Core Capabilities                                                                                                 |
|---------|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| Student | End user (learner)       | Take assessment, view roadmap, chat with AI, request/accept mentor match, upgrade to Premium, download PDF report |
| Admin   | Single platform operator | Manage mentor directory, assign/override mentor matches, view revenue & usage stats                               |

### 2.3 Current Build Status

Grounded in the existing repository, before any of the work in this spec begins:

| Layer                                           | Status                                        |
|-------------------------------------------------|-----------------------------------------------|
| Auth (Supabase sign-in / sign-up)               | <b>Built</b>                                  |
| Global providers (Auth, Theme), Header, Toaster | <b>Built</b>                                  |
| Base UI kit (Button, Card, Input, Label, Toast) | <b>Built</b>                                  |
| Design tokens (OKLCH, Tailwind v4)              | <b>Built</b>                                  |
| Homepage                                        | <b>Pending</b> -- currently a placeholder div |
| Prisma schema                                   | <b>Pending</b> -- no models defined yet       |
| All AI functionality                            | <b>Pending</b>                                |
| Assessment, Roadmap, Chatbot, Account pages     | <b>Pending</b>                                |
| Admin panel                                     | <b>Pending</b>                                |
| Route protection (RBAC)                         | <b>Pending</b>                                |
| Payments                                        | <b>Pending</b>                                |

### 3. Technology Stack

#### 3.1 Core Framework (already decided)

| Technology | Version | Role                                    |
|------------|---------|-----------------------------------------|
| Next.js    | 16.3.1  | Full-stack React framework (App Router) |
| React      | 19.2.8  | UI library                              |
| TypeScript | ^5      | Type-safe JavaScript                    |

#### 3.2 Styling & UI System (already decided)

| Technology               | Version         | Role                                 |
|--------------------------|-----------------|--------------------------------------|
| Tailwind CSS             | ^4              | Utility-first CSS, new config system |
| shadcn/ui                | ^4.18.0         | Component scaffolding system         |
| Base UI (@base-ui/react) | ^1.7.0          | Headless UI primitives               |
| class-variance-authority | ^0.7.1          | Typed component variants             |
| clsx + tailwind-merge    | ^2.1.1 / ^3.6.0 | Conditional class merging            |
| Hugeicons                | ^1.1.9          | Icon library                         |

#### 3.3 Backend & Data Layer (already decided)

| Technology    | Version  | Role                                          |
|---------------|----------|-----------------------------------------------|
| Supabase      | ^2.112.3 | Auth + hosted PostgreSQL (AWS ap-southeast-1) |
| @supabase/ssr | ^0.12.4  | SSR-compatible Supabase client helpers        |
| Prisma        | ^7.9.1   | ORM for schema, migrations, queries           |

#### 3.4 AI Layer (new -- required for this spec)

| Package            | Purpose                                                                     |
|--------------------|-----------------------------------------------------------------------------|
| ai (Vercel AI SDK) | Unified model calling, streaming chat, generateObject for structured output |
| @ai-sdk/google     | Gemini provider for the Vercel AI SDK                                       |
| zod                | Schema validation -- both AI structured output and form validation          |

**Recommendation:** Gemini 2.0 Flash via the Vercel AI SDK, using generateObject with a Zod schema. Rationale: strong structured-output support, generous free tier, and provider-agnostic SDK means switching to OpenAI or Anthropic later is a one-line change. See Section 16 for this as a flagged assumption.

### 3.5 Automation & Integrations (new)

| Package             | Purpose                                                                   |
|---------------------|---------------------------------------------------------------------------|
| react-hook-form     | Form state management -- assessment stepper, auth forms                   |
| @hookform/resolvers | Zod resolver bridge for react-hook-form                                   |
| @react-pdf/renderer | Server-side, serverless-friendly PDF report generation                    |
| razorpay            | Payment gateway Node SDK (test mode for demo)                             |
| framer-motion       | Scroll-narrative landing page, robot assistant motion, micro-interactions |
| recharts            | Admin revenue & usage charts                                              |

Exact patch versions aren't pinned here -- run `npm view <package> version` before installing to confirm the current major version of each.

### 3.6 Tooling (already decided)

| Tool                                      | Role                                             |
|-------------------------------------------|--------------------------------------------------|
| ESLint v9 (flat config)                   | Linting -- eslint-config-next + TypeScript rules |
| Prettier v3 + prettier-plugin-tailwindcss | Code formatting, auto-sorted Tailwind classes    |
| babel-plugin-react-compiler               | React Compiler support (experimental)            |



## 4. Information Architecture -- Pages & Layouts

**Direct answer:** 11 distinct pages/routes, organized under 4 layout templates (Root, Marketing, App Shell, Admin Shell).

### 4.1 Layout Templates

| Layout       | Wraps                                         | Contains                                                                    |
|--------------|-----------------------------------------------|-----------------------------------------------------------------------------|
| Root (built) | Every page                                    | Fonts (Raleway/Outfit), ThemeProvider, AuthProvider, global Header, Toaster |
| Marketing    | Landing, Auth, 404                            | Minimal chrome -- public header/footer, no sidebar                          |
| App Shell    | Dashboard, Assessment, Roadmap, Chat, Account | Sidebar/topnav, persistent Robot Assistant widget, voice command bar        |
| Admin Shell  | Admin Dashboard, Mentors, Revenue             | Separate admin sidebar nav, distinct from the student app shell             |

### 4.2 Full Route Inventory

| #  | Page                   | Route          | Layout      | Role    | Status  |
|----|------------------------|----------------|-------------|---------|---------|
| 1  | Landing                | /              | Marketing   | Public  | Pending |
| 2  | Auth                   | /auth          | Marketing   | Public  | Built   |
| 3  | Dashboard              | /dashboard     | App Shell   | Student | Pending |
| 4  | Assessment             | /assessment    | App Shell   | Student | Pending |
| 5  | Career Paths / Roadmap | /roadmap       | App Shell   | Student | Pending |
| 6  | AI Chatbot             | /chat          | App Shell   | Student | Pending |
| 7  | Account                | /account       | App Shell   | Student | Pending |
| 8  | Admin Dashboard        | /admin         | Admin Shell | Admin   | Pending |
| 9  | Admin Mentors          | /admin/mentors | Admin Shell | Admin   | Pending |
| 10 | Admin Revenue          | /admin/revenue | Admin Shell | Admin   | Pending |
| 11 | Not Found              | /* (404)       | Marketing   | Public  | Pending |

### 4.3 Page-by-Page Breakdown

#### Landing Page -- /

- Hero: headline, subheadline, idle-state robot mascot, primary CTA ("Start Free Assessment")
- Scroll-narrative section: 4-5 scroll-revealed panels telling one student's story -- unsure about a career, takes the assessment, receives an AI roadmap, gets a mentor and prepares for exams, is placed in college. Built with Framer Motion whileInView triggers per panel (simpler and more robust than scroll-pinned animation for

this timeline).

- Premium pitch: Free vs Premium plan comparison (reuses Card component)
- Footer CTA into /auth

### **Auth -- /auth (built)**

Dual-mode sign-in/sign-up, already functional. No changes required beyond ensuring a default role of STUDENT is set on new signups (see Section 9).

### **Dashboard -- /dashboard**

- Robot Assistant widget: persistent, cursor-tracking, reflects idle/listening/thinking/speaking state (Section 5.5)
- Quick-status cards: assessment status, roadmap progress, mentor assigned (yes/no)
- Voice command bar: mic icon, Web Speech API (Section 13)
- Quick links into Assessment, Roadmap, Chat, Account

### **Assessment -- /assessment**

- Multi-step form (react-hook-form + Zod resolver): education background, stream, skills (multi-select), interests (multi-select), personality, freetext goals
- Progress indicator across steps
- On submit: POST /api/assessment/submit, which triggers AI generation and redirects to /roadmap once complete

### **Career Paths / Roadmap -- /roadmap**

- AI-generated summary paragraph
- 2-4 expandable Career Path cards: steps, recommended exams + alternates, matched universities, competition-level badge, future-demand badge, hiring companies
- "Download PDF Report" button (Section 8.1)
- "Request Mentor" button -- surfaces auto-suggested matches or lets the student flag interest for admin to assign manually

### **AI Chatbot -- persistent panel + /chat**

- Available as a slide-over panel from any App Shell page, plus a dedicated /chat page for full history
- Mode indicator badge: General or Personalized (Section 7.3)
- Streaming responses via the Vercel AI SDK useChat hook

### **Account -- /account**

- Profile info, subscription tier + upgrade CTA
- Assessment history list
- Assigned mentor card, if any

### **Admin Dashboard -- /admin**

- Stat cards: total students, total mentors, unassigned students, monthly revenue

### **Admin Mentors -- /admin/mentors**

- Mentor table: add/edit mentor, field-specialization tags

- Unassigned-students list with ranked auto-match suggestions and a manual override dropdown

#### **Admin Revenue -- /admin/revenue**

- Revenue chart (Recharts, monthly)
- Payment transaction table

## 5. Design System & Styling

### 5.1 Styling Approach (already decided)

Tailwind CSS v4 with the new CSS-first config system -- no tailwind.config.js. Design tokens are defined as CSS custom properties in src/styles/globals.css using the OKLCH color space, mapped to Tailwind utilities via an @theme inline block.

### 5.2 Typography

| Font    | CSS Variable   | Usage                             |
|---------|----------------|-----------------------------------|
| Raleway | --font-sans    | Body text, paragraphs             |
| Outfit  | --font-heading | Headings, card titles, brand name |

Both are loaded via next/font/google for zero layout shift and self-hosting.

### 5.3 Core Color Tokens (light mode)

| Token         | OKLCH Value                | Description                |
|---------------|----------------------------|----------------------------|
| --background  | oklch(1 0 0)               | Pure white                 |
| --foreground  | oklch(0.148 0.004 228.8)   | Near-black, cool navy tint |
| --primary     | oklch(0.488 0.243 264.376) | Deep indigo/violet         |
| --muted       | oklch(0.963 0.002 197.1)   | Very light cool grey       |
| --destructive | oklch(0.577 0.245 27.325)  | Deep red/orange            |
| --border      | oklch(0.925 0.005 214.3)   | Subtle cool grey           |

Dark-mode equivalents live in the .dark class in globals.css. Border radius scale is --radius: 0.625rem base, exposed as --radius-sm through --radius-4xl; UI components default to rounded-2xl for a modern card-based look.

### 5.4 Component Library (already built)

shadcn/ui + Base UI primitives: Button (6 variants, 8 sizes), Card (compound pattern with Header/Title/Description/Action/Content/Footer), Input, Label, and a full Toast system (success/info/warning/error/loading). New feature work should compose from these rather than introducing a second component style.

### 5.5 The Robot Assistant -- Technical Implementation

The recurring UI idea across planning sketches: a robot mascot whose eyes track the cursor and which visually reflects what the app is doing.

- Rendered as an SVG (or Lottie JSON if a design asset is supplied) React component, RobotAssistant.tsx
- Cursor tracking: a document-level mousemove listener computes the angle from each eye's center to the pointer using atan2(pointerY - eyeY, pointerX - eyeX), then applies a clamped rotation/translation to the pupil elements so they never leave the eye socket
- Implemented as a small custom hook, useCursorTracking(), reused by both eyes

- Four visual states driven by app context: idle (subtle blink loop), listening (voice command mic engaged), thinking (pulse while an AI request is in flight), speaking (mouth animation while a chatbot response streams in)
- State transitions animated with Framer Motion's animate prop, keyed off a single state prop passed down from Dashboard / Chat

## 5.6 Motion & Animation Strategy

Framer Motion is the single animation library for the whole app -- scroll-triggered landing page panels (whileInView), the robot assistant, and general micro-interactions (hover/tap states on cards and buttons). Avoid mixing in a second animation library; it adds bundle size without added capability here.

## 5.7 Responsive Strategy

Tailwind's default breakpoints (sm/md/lg/xl/2xl) apply throughout. Priorities: the Assessment stepper and Chat panel must be fully usable at mobile widths first (these are the highest-traffic student screens); the Admin Shell can assume tablet-or-larger since it has a single, non-public user.

## 6. Database Architecture

### 6.1 Entity Overview

A User has many Assessments; each Assessment produces multiple CareerPaths. A User may have one MentorAssignment, linking to a Mentor. Payments and ChatSessions (with nested ChatMessages) both belong to a User. Role and subscription tier live directly on User.

### 6.2 Full Prisma Schema

```
// -- Enums -----
enum Role {
  STUDENT
  ADMIN
}

enum SubscriptionTier {
  FREE
  PREMIUM
}

enum MatchType {
  MANUAL
  AUTO
}

// -- Core Models -----
model User {
  id          String      @id @default(uuid())
  supabaseId  String      @unique
  email       String      @unique
  name        String?
  role        Role         @default(STUDENT)
  subscriptionTier SubscriptionTier @default(FREE)
  aiTokensUsed Int         @default(0)
  aiTokensLimit Int        @default(50)
  createdAt   DateTime     @default(now())

  assessments Assessment[]
  chatSessions ChatSession[]
  mentorAssignment MentorAssignment?
  payments     Payment[]
}
```

```

}

model Assessment {
  id          String      @id @default(uuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
  createdAt   DateTime     @default(now())

  education   String?
  stream      String?
  skills      String[]
  interests   String[]
  personality  String?
  goals       String?

  recommendation Json?
  pdfReportUrl  String?
  careerPaths   CareerPath[]
}

model CareerPath {
  id          String      @id @default(uuid())
  assessmentId String
  assessment   Assessment @relation(fields: [assessmentId], references: [id])

  title          String
  description     String
  steps          Json
  recommendedExams String[]
  alternateExams String[]
  universities    Json
  competitionLevel String?
}

```

```

    futureDemand      String?
    companies          String[]
  }

// -- Mentor System -----
model Mentor {
  id          String      @id @default(uuid())
  name        String
  fieldSpecialization String[]
  bio         String?
  organization String?
  createdAt   DateTime    @default(now())

  assignments      MentorAssignment[]
}

model MentorAssignment {
  id          String      @id @default(uuid())
  studentId   String      @unique
  student     User        @relation(fields: [studentId], references: [id])
  mentorId    String
  mentor      Mentor      @relation(fields: [mentorId], references: [id])
  matchType   MatchType
  assignedAt  DateTime    @default(now())
}

// -- Payments -----
model Payment {
  id          String      @id @default(uuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
  razorpayOrderId String

```

```

  amount      Int
  currency    String      @default("INR")
  status      String
  createdAt   DateTime    @default(now())
}

// -- AI Chatbot -----
model ChatSession {
  id          String      @id @default(uuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
  mode        String
  createdAt   DateTime    @default(now())

  messages    ChatMessage[]
}

model ChatMessage {
  id          String      @id @default(uuid())
  sessionId   String
  session     ChatSession @relation(fields: [sessionId], references: [id])
  role        String
  content     String
  createdAt   DateTime    @default(now())
}

```

### 6.3 Field-Level Notes

- User.role drives all route protection (Section 9) and is the STUDENT/ADMIN switch



- User.aiTokensUsed / aiTokensLimit implement the free-vs-premium AI usage gate; premium checks bypass the limit rather than raising it to a very large number, keeping the logic explicit
- CareerPath.recommendedExams / alternateExams / universities / competitionLevel / futureDemand / companies map directly to the roadmap fields described in planning -- exam guidance (e.g. JEE Advanced with CAT as an alternate route), matched universities, and market outlook
- MentorAssignment.matchType records whether a pairing was AUTO-suggested and accepted, or MANUAL -- useful later for measuring how good the auto-match heuristic actually is

## 7. AI System Architecture

### 7.1 Two AI Touchpoints

The system has exactly two places AI is called: the one-time Assessment Recommendation Engine, and the ongoing AI Chatbot, which runs in two modes as specified.

### 7.2 Assessment Recommendation Engine

#### Flow

1. Student submits the multi-step form; client validates via the same Zod schema used server-side
2. POST /api/assessment/submit writes the raw Assessment record (education, stream, skills, interests, personality, goals)
3. Server builds a prompt combining the student's inputs with system instructions (Indian education context, structured-output requirement)
4. Calls Gemini 2.0 Flash via the Vercel AI SDK's generateObject, passing the Zod schema below
5. The validated structured result is saved to Assessment.recommendation and expanded into individual CareerPath rows
6. User.aiTokensUsed is incremented
7. Student is redirected to /roadmap; a mentor auto-match suggestion is generated asynchronously for the admin queue

#### Output Schema

```
import { z } from "zod";

export const CareerPathSchema = z.object({
  title: z.string(),
  description: z.string(),
  steps: z.array(
    z.object({
      order: z.number(),
      title: z.string(),
      description: z.string(),
    })
  ),
  recommendedExams: z.array(z.string()),
  alternateExams: z.array(z.string()),
  universities: z.array(
    z.object({
      name: z.string(),
      courses: z.array(z.string()),
    })
  ),
  competitionLevel: z.enum(["Low", "Moderate", "High", "Very High"]),
  futureDemand: z.enum(["Declining", "Stable", "Growing", "High Growth"]),
  companies: z.array(z.string()),
});

export const AssessmentRecommendationSchema = z.object({
  summary: z.string(),
  careerPaths: z.array(CareerPathSchema).min(2).max(4),
});
```

#### Generation Call

```
const result = await generateObject({
  model: google("gemini-2.0-flash"),
  schema: AssessmentRecommendationSchema,
  system: ASSESSMENT_SYSTEM_PROMPT,
  prompt: buildAssessmentPrompt(studentInput),
});

await prisma.assessment.update({
  where: { id: assessmentId },
  data: { recommendation: result.object },
});
```

### 7.3 AI Chatbot -- Dual-Mode Design

As specified: general career guidance before a student has an assessment on file, personalized and roadmap-aware after.

#### General Mode (pre-assessment)

System prompt positions the assistant as a general Indian career-guidance helper: encouraging, broadly informative, and explicit that it will get much more specific once the student completes the assessment.

#### Personalized Mode (post-assessment)

The server looks up the student's most recent Assessment and its CareerPaths, and injects the key fields into the system prompt so the model can reference specifics -- exam names, the chosen path's title, stated goals -- rather than speaking generically.

#### Mode-Switch Logic

```
async function buildSystemPrompt(userId: string): Promise<string> {
  const user = await prisma.user.findUnique({
    where: { id: userId },
    include: {
      assessments: {
        orderBy: { createdAt: "desc" },
        take: 1,
        include: { careerPaths: true },
      },
    },
  });

  if (!user || user.assessments.length === 0) {
    return GENERAL_CAREER_SYSTEM_PROMPT;
  }

  const latest = user.assessments[0];
  const paths = latest.careerPaths.map((c) => c.title).join(", ");

  return PERSONALIZED_SYSTEM_PROMPT
    .replace("{{stream}}", latest.stream ?? "not specified")
    .replace("{{careerPaths}}", paths)
    .replace("{{goals}}", latest.goals ?? "not specified");
}
```

ChatSession.mode is stamped at session creation from this same check, so the history view (Section 4.3) can label past sessions General or Personalized.

### 7.4 Token & Usage Management

Every assessment generation and every chat message increments `User.aiTokensUsed`. FREE-tier requests are blocked once `aiTokensUsed` reaches `aiTokensLimit` (default 50); PREMIUM requests skip the check entirely. This mirrors the "unlimited assessments, more AI tokens" premium benefit from planning.

## 8. Automation Systems

### 8.1 Automated PDF Report Generation

- Trigger: automatically right after an assessment's AI recommendation is saved, and on-demand from the "Download PDF Report" button on /roadmap
- Library: @react-pdf/renderer, chosen over a Puppeteer/headless-browser approach because it has no binary dependency and works cleanly on Vercel's serverless functions
- Flow: /api/reports/generate fetches the Assessment + CareerPaths + user info, renders them through a React ReportDocument template (Page/Text/View components from @react-pdf/renderer), and either streams the PDF back or uploads it to Supabase Storage and saves the URL to Assessment.pdfReportUrl
- Report contents: student summary, each career path with steps, exam guidance, matched universities, and the AI-generated summary paragraph

### 8.2 Mentor Matching Automation

Hybrid, as specified: manual assignment is the baseline; auto-match is an additive suggestion layer the admin can accept or override.

#### Scoring

```
function scoreMentor(paths: CareerPath[], mentor: Mentor): number {
  const tags = new Set(
    paths.flatMap((p) => [p.title, ...p.companies]).map((s) => s.toLowerCase())
  );
  return mentor.fieldSpecialization.filter((f) =>
    tags.has(f.toLowerCase())
  ).length;
}

// Rank mentors by overlap; top 3 shown to admin as suggestions
const ranked = mentors
  .map((m) => ({ mentor: m, score: scoreMentor(studentPaths, m) }))
  .sort((a, b) => b.score - a.score)
  .slice(0, 3);
```

This is deliberately a simple tag-overlap heuristic rather than an ML model -- appropriate for the mentor-pool size at this stage, and easy to reason about when an admin asks "why was this mentor suggested?" It can be swapped for something more sophisticated later without changing the MentorAssignment data model.

#### Hybrid UX

Admin sees ranked auto-suggestions per unassigned student and can accept one (creates a MentorAssignment with matchType AUTO) or pick any mentor manually (matchType MANUAL).

### 8.3 Session Refresh Automation (already built)

src/proxy.ts middleware already refreshes the Supabase session on every request following the official Supabase SSR pattern. No changes needed beyond the RBAC extension in Section 9.

## 9. Role-Based Access Control

### 9.1 Role Storage Strategy

Role is stored in two places by design: in Supabase `user_metadata` (fast, edge-readable, used by middleware for routing decisions) and in Prisma's `User.role` (source of truth for all application logic). New signups default to `STUDENT` in both places; promoting the single Admin account is a manual, one-time operation.

### 9.2 Route Protection Matrix

| Route                                              | Required Role | If Unauthenticated | If Wrong Role                     |
|----------------------------------------------------|---------------|--------------------|-----------------------------------|
| / , /auth                                          | Public        | Allow              | n/a                               |
| /dashboard, /assessment, /roadmap, /chat, /account | STUDENT       | Redirect to /auth  | ADMIN -> redirect to /admin       |
| /admin/*                                           | ADMIN         | Redirect to /auth  | STUDENT -> redirect to /dashboard |

### 9.3 Middleware Flow

`src/proxy.ts` is extended, after the existing Supabase session refresh, with a lookup of the role from the (now-refreshed) session's `user_metadata`, then applies the matrix above before letting the request through to the page.

## 10. Payments & Premium System

**Test mode only.** Use Razorpay test API keys (rzp\_test\_...) for the entire build and demo. Never wire live/production keys into a hackathon demo environment -- this avoids taking real money from anyone testing the flow.

### 10.1 Razorpay Integration Flow

- 1. Student clicks "Upgrade to Premium"
- 2. POST /api/payments/create-order -- server creates a Razorpay order in test mode (amount in paise)
- 3. Client opens Razorpay Checkout with the returned order ID and the test key
- 4. On success, Razorpay returns a payment ID and signature to the client
- 5. POST /api/payments/verify -- server verifies the HMAC signature against the test secret
- 6. If valid: create a Payment record and set User.subscriptionTier to PREMIUM

### 10.2 Premium Feature Gate

| Feature                          | Free                     | Premium                 |
|----------------------------------|--------------------------|-------------------------|
| Assessments                      | Limited by aiTokensLimit | Unlimited               |
| AI chatbot messages              | Limited by aiTokensLimit | Unlimited               |
| Full syllabus access             | Not included             | Included                |
| India college syllabus alignment | Not included             | Included                |
| Personal mentorship throughout   | Not included             | Included                |
| Course catalog                   | Core courses             | Extended course catalog |

### 10.3 Security Notes

- Signature verification happens server-side only -- never trust a client-reported "payment succeeded" without HMAC verification
- Razorpay test/live key pairs are environment-specific; keep test keys in .env.local and never commit either set

## 11. API Route Inventory

| Method | Route                            | Purpose                                    | Auth            |
|--------|----------------------------------|--------------------------------------------|-----------------|
| POST   | /api/assessment/submit           | Submit assessment, trigger AI generation   | Student         |
| GET    | /api/assessment/[id]             | Fetch an assessment + its roadmap          | Student (owner) |
| POST   | /api/reports/generate            | Generate / fetch the PDF report            | Student (owner) |
| POST   | /api/chat                        | Streaming chat message; mode auto-detected | Student         |
| GET    | /api/mentors                     | List mentors                               | Admin           |
| POST   | /api/mentors                     | Create a mentor                            | Admin           |
| POST   | /api/mentors/assign              | Manual mentor assignment                   | Admin           |
| GET    | /api/mentors/suggest/[studentId] | Auto-match suggestions                     | Admin           |
| POST   | /api/payments/create-order       | Create a Razorpay order                    | Student         |
| POST   | /api/payments/verify             | Verify payment, upgrade tier               | Student         |
| GET    | /api/admin/stats                 | Revenue + counts for admin dashboard       | Admin           |



## 12. Folder & File Structure

Extending the existing repository layout with everything introduced in this spec:

```
ai-career-counsellor/
|-- prisma/
|   |-- schema.prisma
|-- src/
|   |-- app/
|   |   |-- layout.tsx
|   |   |-- page.tsx                      (landing -- scroll narrative)
|   |   |-- auth/page.tsx
|   |   |-- dashboard/page.tsx
|   |   |-- assessment/page.tsx
|   |   |-- roadmap/page.tsx
|   |   |-- chat/page.tsx
|   |   |-- account/page.tsx
|   |   |-- admin/
|   |   |   |-- page.tsx                  (admin dashboard)
|   |   |   |-- mentors/page.tsx
|   |   |   |-- revenue/page.tsx
|   |   |-- api/
|   |   |   |-- assessment/
|   |   |   |   |-- submit/route.ts
|   |   |   |   |-- [id]/route.ts
|   |   |   |-- reports/generate/route.ts
|   |   |   |-- chat/route.ts
|   |   |   |-- mentors/
|   |   |   |   |-- route.ts
|   |   |   |   |-- assign/route.ts
|   |   |   |   |-- suggest/[studentId]/route.ts
|   |   |-- payments/
|   |   |   |-- create-order/route.ts
|   |   |   |-- verify/route.ts
|   |   |-- admin/stats/route.ts
|   |
|   |
```

```

|-- components/
|   |-- ui/                                (existing: button, card, input, label, toast, header)
|   |-- robot/
|   |   |-- RobotAssistant.tsx
|   |   |-- useCursorTracking.ts
|   |-- chat/
|   |   |-- ChatPanel.tsx
|   |   |-- ChatMessage.tsx
|   |-- assessment/AssessmentStepper.tsx
|   |-- roadmap/CareerPathCard.tsx
|   |-- admin/
|   |   |-- MentorTable.tsx
|   |   |-- RevenueChart.tsx
|   |-- voice/VoiceCommandBar.tsx
|
|-- lib/
|   |-- utils.ts                          (existing)
|   |-- prisma.ts                        (Prisma client singleton)
|   |-- ai/
|   |   |-- schemas.ts                  (Zod schemas)
|   |   |-- prompts.ts                 (system prompt templates)
|   |   |-- client.ts                  (Vercel AI SDK + Gemini setup)
|   |-- pdf/ReportDocument.tsx          (@react-pdf/renderer template)
|   |-- matching/scoreMentor.ts
|   |-- razorpay/client.ts
|
|-- providers/                          (existing: AuthProvider, ThemeProvider)
|-- proxy.ts                            (extended with RBAC route guard)
|-- utils/supabase/                    (existing)

```

## 13. Voice Command System

The dashboard sketch called for voice-driven navigation. Implemented with the browser-native Web Speech API rather than a third-party service -- zero added cost and no extra network dependency, at the cost of inconsistent browser support (strong in Chrome/Edge, partial in Safari, weak in Firefox). The mic button feature-detects window.SpeechRecognition and hides itself where unsupported.

### Intent Mapping

A useVoiceCommand() hook starts listening on mic tap, and matches the returned transcript against a lightweight keyword table -- no NLU service required at this scale:

| Phrase contains              | Action                  |
|------------------------------|-------------------------|
| "assessment" / "start test"  | Navigate to /assessment |
| "roadmap" / "career path"    | Navigate to /roadmap    |
| "chat" / "talk to assistant" | Open the chatbot panel  |
| "account" / "profile"        | Navigate to /account    |

## 14. Development Roadmap

---

No hard external deadline is assumed for this ordering -- phases are sequenced by technical dependency, not by day count. Each phase should be functionally complete before the next one starts.

### Phase 1 -- Data Layer Foundation

- Write the full Prisma schema (Section 6.2) into `prisma/schema.prisma`
- Run the initial migration (`npx prisma migrate dev --name init`)
- Seed script with a handful of sample mentors for local testing
- Default role to STUDENT in Supabase `user_metadata` on signup

### Phase 2 -- AI Assessment Engine

- Install `ai`, `@ai-sdk/google`, `zod`
- Define the Zod schemas from Section 7.2
- Write system + user prompt templates
- Build `/api/assessment/submit` and test structured output against several sample inputs

### Phase 3 -- Student Core Loop

- Build the Assessment multi-step form
- Build the Roadmap / Career Paths results page
- Build a static-first Dashboard shell (robot/voice come later, in Phase 8)
- Wire Assessment -> Roadmap navigation end to end

### Phase 4 -- Automation Layer

- Install `@react-pdf/renderer`, build the report template and `/api/reports/generate`
- Build the mentor scoring function and `/api/mentors/suggest/[studentId]`

### Phase 5 -- Admin Panel

- Admin dashboard, mentor CRUD, manual + auto-suggested assignment UI
- Revenue page (placeholder data first; wired to real payments in Phase 7)

### Phase 6 -- AI Chatbot (Dual Mode)

- `ChatSession` / `ChatMessage` wiring
- `/api/chat` streaming route with mode-detection logic (Section 7.3)
- Chat panel UI (`useChat` hook) and the dedicated `/chat` history page

### Phase 7 -- Payments & Premium Gating

- Set up a Razorpay test-mode account and keys
- `/api/payments/create-order` and `/api/payments/verify`
- Wire the feature gates from Section 10.2

### Phase 8 -- Landing Page & Motion Polish

- Scroll-narrative sections (Framer Motion)
- Premium pitch section

- Robot Assistant -- cursor tracking and state machine, wired into both Landing and Dashboard

### **Phase 9 -- RBAC Hardening + Voice Commands**

- Extend proxy.ts with the full route guard matrix (Section 9.2)
- Web Speech API voice command bar and intent mapping (Section 13)

### **Phase 10 -- QA & Deploy**

- Error/404 pages, mobile responsiveness pass
- Deploy to Vercel, verify environment variables, run the Razorpay test flow end to end

## 15. Developer Commands

| Command                    | Description                                           |
|----------------------------|-------------------------------------------------------|
| npm run dev                | Start the development server at http://localhost:3000 |
| npm run build              | Build the production bundle                           |
| npm run lint               | Run ESLint on the codebase                            |
| npx prisma studio          | Open Prisma's visual database browser                 |
| npx prisma migrate dev     | Create and apply a new database migration             |
| npx prisma generate        | Regenerate the Prisma Client after schema changes     |
| npx prisma db seed         | Run the seed script (mentors, sample data)            |
| npx shadcn add [component] | Add a new shadcn UI component                         |

## 16. Appendix -- Assumptions & Open Decisions

---

Every judgment call made in this document that wasn't explicitly specified, so it can be revisited before implementation begins:

- **AI provider:** Gemini 2.0 Flash via the Vercel AI SDK. OpenAI (GPT-4o-mini) remains a drop-in fallback if Gemini's structured-output quality or quota doesn't hold up in testing.
- **Payment gateway:** Razorpay, TEST MODE ONLY. Chosen for native India/UPI support and SIH context. Must not be switched to live keys without a separate compliance and security review.
- **PDF generation:** @react-pdf/renderer over a Puppeteer/headless-browser approach, specifically because Vercel's serverless functions handle it without a browser binary.
- **Charts:** Recharts, since it pairs natively with shadcn/ui's chart components.
- **Voice input:** native Web Speech API, not a third-party speech service -- zero incremental cost, at the cost of inconsistent cross-browser support (documented in Section 13).
- **Mentor auto-match:** a simple tag-overlap score, not a machine-learning model -- appropriate for the expected mentor-pool size, and easy for an admin to audit.
- **Robot Assistant asset:** spec assumes a hand-built SVG unless the design/asset team supplies a Lottie animation, in which case swap the rendering layer only -- the state machine and cursor-tracking hook stay the same.